



MentorMatch

Università degli Studi dell'Insubria – Laurea Triennale in Informatica

TECNOLOGIE INNOVATIVE PER LO SVILUPPO WEB

Creato da: Mattia Gerundino, Colombo Patrizio, Felitti Fabio, Iacono Alessandro.



SOMMARIO

DevOps & Cloud	2
Obiettivo	2
Struttura del progetto	2
Preparazione backend (Express)	2
Gestione Stripe resiliente	3
Dockerizzazione	3
Test locale	4
CI con GitHub Actions	4
Deploy su Render (CD)	5
Continuous Deployment	5
Monitoraggio	5
Scaling	5
Conclusione	5
schema ER	6
API	6
Panoramica	6
Autenticazione	7
Gestione Profilo Utente	7
Prenotazioni (Bookings)	8
Gestione Sessioni (Disponibilità Mentor)	8
Messaggistica	9
Mentor e Ricerca	9
Recensioni	10
Gestione Account	10
Gestione errori	10



DEVOPS & CLOUD

Questo documento riassume passo passo tutte le attività DevOps/Cloud svolte per il backend del progetto MentorMatch: Dockerizzazione, CI/CD, deploy cloud, monitoraggio e scaling.

OBIETTIVO

Implementare una pipeline DevOps completa per il backend Express: - Containerizzazione con Docker Continuous Integration con GitHub Actions - Continuous Deployment su Render - Monitoraggio tramite health check e log - Preparazione allo scaling automatico

STRUTTURA DEL PROGETTO

```
MentorMatch/
├── backend/
│   ├── Dockerfile
│   ├── .dockerignore
│   ├── package.json
│   ├── src/
│   │   └── app.js
│   └── controllers/
│       └── stripe_webhook_controller.js
└── .github/
    ├── workflows/
    └── docker-ci.yml
```

PREPARAZIONE BACKEND (EXPRESS)

Porta dinamica e binding

```
In backend/src/app.js:
    const PORT = process.env.PORT || 3000;
    app.listen(PORT, "0.0.0.0", () => {
        console.log(`Server listening on port ${PORT}`);
    });
```

Health check

```
app.get("/health", (req, res) => res.status(200).send("ok"));
```

Log richieste

```
app.use((req, res, next) => {
    console.log(`${req.method} ${req.path}`);
    next();
});
```



GESTIONE STRIPE RESILIENTE

In controllers/stripe_webhook_controller.js:

```
const STRIPE_KEY = process.env.STRIPE_SECRET_KEY;
const WEBHOOK_SECRET = process.env.STRIPE_WEBHOOK_SECRET;

const stripe = STRIPE_KEY ? new Stripe(STRIPE_KEY) : null;

export const stripeWebHookController = async (req, res) => {
  if (!stripe || !WEBHOOK_SECRET) {
    return res.status(503).json({
      error: "Stripe non configurato"
    });
  }
  // logica webhook...
};
```

In questo modo: - L'app non crasha se mancano le variabili d'ambiente - La CI/CD può avviarsi senza segreti

DOCKERIZZAZIONE

Dockerfile (backend/Dockerfile)

FROM node:20-bookworm-slim

WORKDIR /app

COPY package*.json ./

RUN npm install

COPY . .

EXPOSE 3000

CMD ["npm", "start"]

.dockerignore (backend/.dockerignore)

node_modules

.git

.env

npm-debug.log



TEST LOCALE

```
cd backend
npm install
npm start
```

Test Docker:

```
docker build -t mentor-match .
docker run -p 3000:3000 -e PORT=3000 mentor-match
```

CI CON GITHUB ACTIONS

File `.github/workflows/docker-ci.yml` :

```
name: Docker CI
on:
  push:
    branches: [ "main", "devops", "devOps", "devops-ci" ]
  pull_request:
jobs:
  docker-build-and-smoke-test:
    runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - name: Build Docker image
      run: docker build -t mentor-match ./backend
    - name: Run container
      run: docker run -d --name mentor-match -p 3000:3000 -e PORT=3000
mentor-match
  - name: Smoke test
    run: |
      for i in {1..60}; do
        if curl -sS http://localhost:3000/health > /dev/null; then
          echo "Server responding"
          exit 0
        fi
        sleep 2
      done
    docker logs mentor-match
    exit 1
```



DEPLOY SU RENDER (CD)

Configurazione

- Service type: Web Service
- Root Directory: backend
- Environment: Docker
- Branch: main (o devOps)

Variabili d'ambiente

- NODE_ENV=production
- STRIPE_SECRET_KEY=...
- STRIPE_WEBHOOK_SECRET=...
- DATABASE_URL=...

Test

`https://<servizio>.onrender.com/health`

Risposta attesa:

ok

CONTINUOUS DEPLOYMENT

Render è collegato al repository GitHub: - Ogni push sul branch configurato attiva un nuovo deploy - Build → Start → Messa online automatica

MONITORAGGIO

- Endpoint /health per liveness/readiness
- Log applicativi visibili nella dashboard Render
- Storico deploy disponibile

SCALING

- Gestione risorse demandata al provider cloud (Render)
- Possibilità di scaling verticale (CPU/RAM)
- Possibilità di scaling orizzontale (istanze multiple)
- Health check usato per readiness

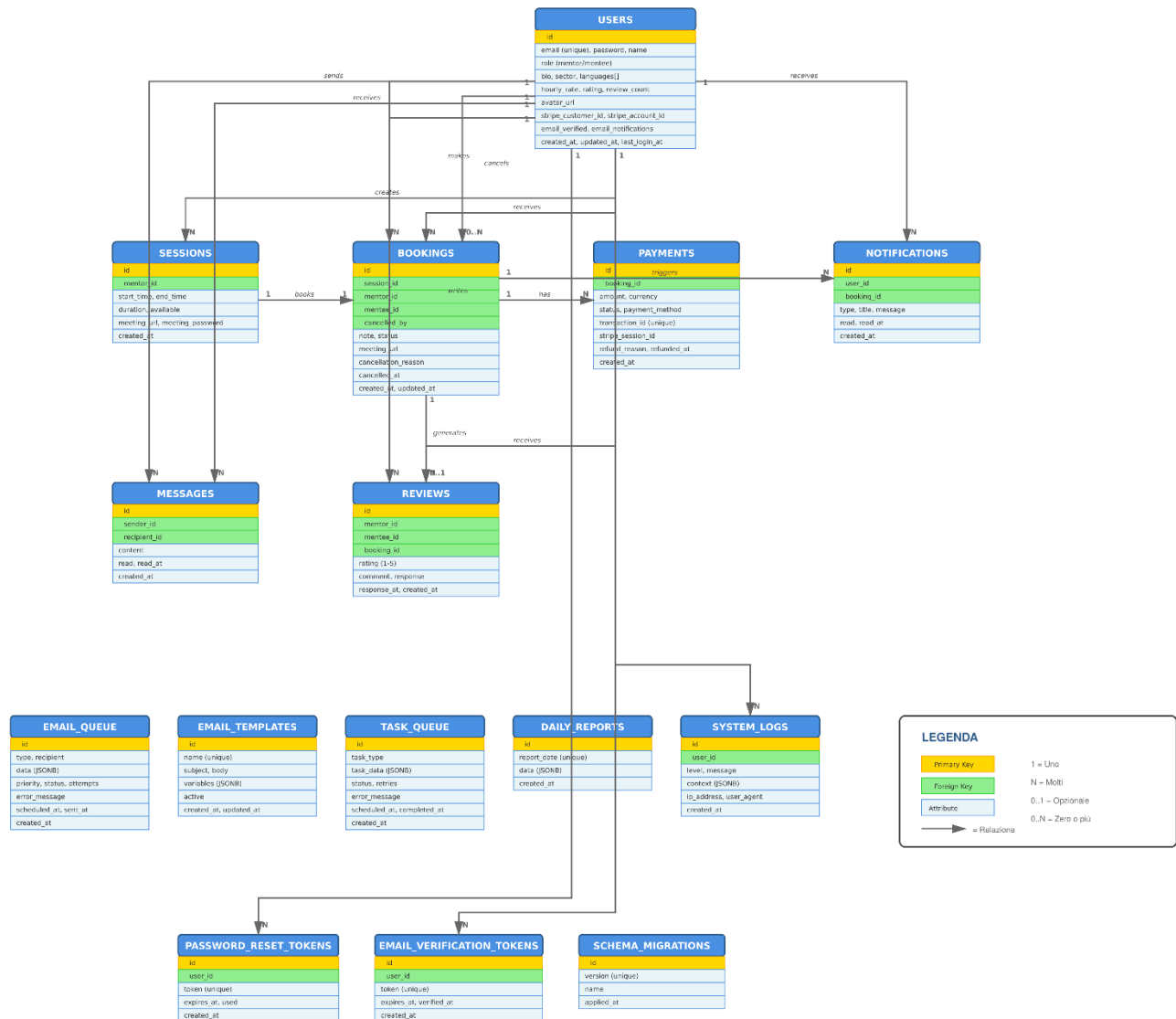
CONCLUSIONE

È stata implementata una pipeline DevOps completa: - Docker per containerizzazione - GitHub Actions per CI - Render per CD - Health check e log per monitoraggio - Architettura pronta per autoscaling

Questo garantisce: - Deploy automatici - Avvio affidabile - Isolamento tramite container - Osservabilità di base - Scalabilità futura



SCHEMA ER



API

PANORAMICA

ApiService è un **modulo client-side** che centralizza tutte le chiamate HTTP verso il backend dell'applicazione **MentorMatch**.

- **Base URL API:** `const API_BASE_URL = 'https://mentormatch-5pg4.onrender.com/api';`

Tutti gli endpoint vengono costruiti a partire da questo prefisso.

- Comunicazione via **Fetch API**
- Payload in **JSON**
- Autenticazione tramite **Bearer Token** recuperato da `AuthService.getUser().token`



AUTENTICAZIONE

Registrazione utente

Funzione: register(userData)

Metodo: POST

Endpoint: /auth/register

Descrizione: crea un nuovo account utente inviando i dati di registrazione al server.

Login

Funzione: login(username, password)

Metodo: POST

Endpoint: /auth/login

Descrizione: autentica l'utente e restituisce i suoi dati e il token JWT.

Nota: nel codice è presente un login di test hardcoded per:

- username: admin
- password: Password.24

Questo bypassa l'API reale e restituisce un utente fittizio.

Refresh Token

Funzione: refreshToken(userId, token)

Metodo: POST

Endpoint: /auth/refresh-token

Descrizione: richiede un nuovo token quando quello attuale sta per scadere.

GESTIONE PROFILO UTENTE

Aggiornamento profilo

Funzione: updateUser(userData)

Metodo: POST

Endpoint: /users/update_profile

Autenticazione: richiesta

Descrizione: aggiorna informazioni come bio, settore, tariffa e preferenze.

Statistiche utente

Funzione: getUserStats()

Metodo: GET

Endpoint: /users/stats

Autenticazione: richiesta

Descrizione: recupera statistiche personali come:

- numero prenotazioni
- sessioni completate
- sessioni future
- totale speso
- valutazione media



PRENOTAZIONI (BOOKINGS)

Creazione prenotazione

Funzione: `prenoteBooking(bookingData)`

Metodo: POST

Endpoint: `/bookings/checkout`

Autenticazione: richiesta

Descrizione: crea una prenotazione e reindirizza l'utente alla pagina di pagamento (`checkout_url`).

Lista prenotazioni utente

Funzione: `getUserBookings()`

Metodo: POST

Endpoint: `/bookings/user_bookings`

Autenticazione: richiesta

Descrizione: restituisce tutte le prenotazioni dell'utente loggato (mentor o mentee).

Annullamento prenotazione

Funzione: `cancelBooking(bookingId, reason)`

Metodo: POST

Endpoint: `/bookings/cancel_booking`

Autenticazione: richiesta

Descrizione: annulla una prenotazione salvando anche il motivo e l'utente che ha effettuato l'annullamento.

GESTIONE SESSIONI (DISPONIBILITÀ MENTOR)

Creazione slot disponibilità

Funzione: `createSession(sessionData)`

Metodo: POST

Endpoint: `/sessions/createSession`

Autenticazione: richiesta

Descrizione: permette a un mentor di creare uno slot orario disponibile.

Eliminazione slot

Funzione: `deleteSession(sessionId)`

Metodo: DELETE

Endpoint: `/sessions/{sessionId}`

Autenticazione: richiesta

Descrizione: elimina uno slot non ancora prenotato.



MESSAGGISTICA

Invio messaggio

Funzione: postMessage(messageData)

Metodo: POST

Endpoint: /messages/post_message

Autenticazione: richiesta

Descrizione: invia un messaggio a un altro utente.

Recupero messaggi

Funzione: getMessages()

Metodo: POST

Endpoint: /messages/get_messages

Autenticazione: richiesta

Descrizione: recupera l'elenco delle conversazioni o messaggi dell'utente.

Storico chat con un utente

Funzione: getChatHistory(otherUserId)

Metodo: POST

Endpoint: /messages/get_chat_history

Autenticazione: richiesta

Descrizione: restituisce tutti i messaggi tra l'utente loggato e un altro utente specifico.

Segna messaggi come letti

Funzione: markMessageAsRead(otherUserId)

Metodo: POST

Endpoint: /messages/mark-as-read

Autenticazione: richiesta

Descrizione: aggiorna lo stato dei messaggi come "letti".

MENTOR E RICERCA

Ricerca mentor

Funzione: searchMentors(filters)

Metodo: POST

Endpoint: /mentors/search

Descrizione: cerca mentor in base a filtri come settore, prezzo, valutazione, ecc.

Elenco settori

Funzione: getAllSectors()

Metodo: GET

Endpoint: /mentors/get-sector

Descrizione: restituisce la lista dei settori disponibili per autocomplete e filtri.



Profilo pubblico mentor

Funzione: getMentorById(id)

Metodo: GET

Endpoint: /mentors/get-mentor/{id}

Descrizione: recupera le informazioni pubbliche di un mentor.

Sessioni disponibili mentor

Funzione: getMentorSessions(mentorId)

Metodo: GET

Endpoint: /sessions/mentor/{mentorId}

Descrizione: restituisce gli slot disponibili di un mentor.

Recensioni mentor

Funzione: getMentorReviews(mentorId)

Metodo: GET

Endpoint: /mentors/reviews/{mentorId}

Descrizione: recupera le recensioni associate a un mentor.

RECENSIONI**Pubblicazione recensione**

Funzione: postReview(reviewData)

Metodo: POST

Endpoint: /reviews/post_review

Autenticazione: richiesta

Risposta a recensione

Funzione: updateReviewResponse(reviewId, responseText)

Metodo: POST

Endpoint: /reviews/response

Autenticazione: richiesta

GESTIONE ACCOUNT**Eliminazione account**

Funzione: deleteAccount(confirmDeletePassword)

Metodo: DELETE

Endpoint: /users/delete_account

Autenticazione: richiesta

Descrizione: elimina definitivamente l'account utente. Grazie ai vincoli del database, anche dati collegati (sessioni, messaggi, ecc.) vengono rimossi.

GESTIONE ERRORI

- controllano se la risposta HTTP è valida
- leggono eventuali messaggi di errore dal server
- lanciano un errore che può essere gestito dall'interfaccia utente

Gli errori vengono anche stampati nella console del browser per il debug.

